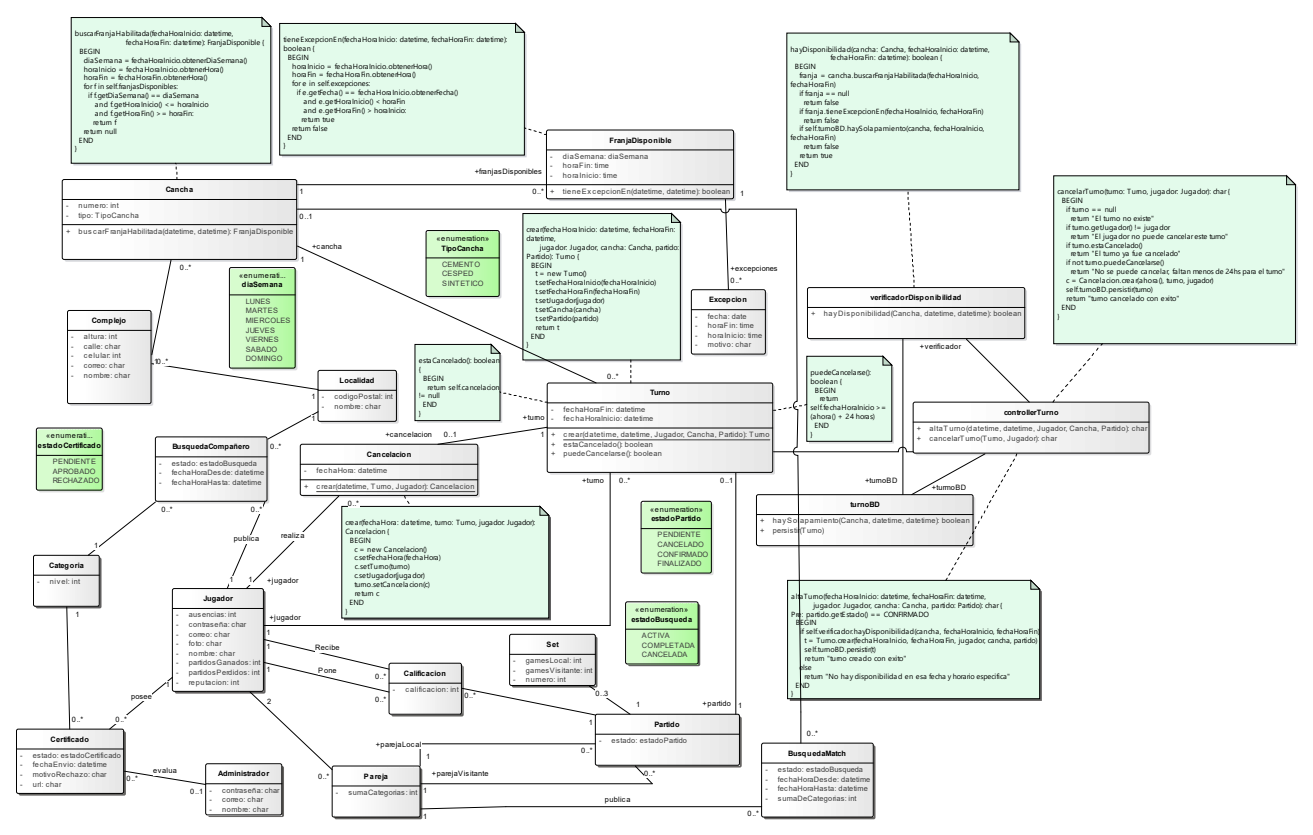

buscarFranjaHabilitada(fechaHoraInicio: datetime, fechaHoraFin: datetime): FranjaDisponible {
BEGIN
diaSemana = fechaHoraInicio.obtenerDiaSemana()
horaInicio = fechaHoraInicio.obtenerHora()
horaFin = fechaHoraFin.obtenerHora()
for f in self.franjasDisponibles:
if f.getDiaSemana() == diaSemana
and f.getHoraInicio() <= horaInicio
and f.getHoraFin() >= horaFin:
return f
return null
END
}
tieneExcepcionEn(fechaHoraInicio: datetime, fechaHoraFin: datetime): boolean {
BEGIN
horaInicio = fechaHoraInicio.obtenerHora()
horaFin = fechaHoraFin.obtenerHora()
for e in self.excepciones:
if e.getFecha() == fechaHoraInicio.obtenerFecha()
and e.getHoraInicio() < horaFin
and e.getHoraFin() > horaInicio:
return true
return false
END
}
FranjaDisponible
- diaSemana: diaSemana
- horaFin: time
- horaInicio: time
+ tieneExcepcionEn(datetime, datetime): boolean
+franjasDisponibles
Cancha
- numero: int
- tipo: TipoCancha
+ buscarFranjaHabilitada(datetime, datetime): FranjaDisponible
+cancha
«enumeration» TipoCancha
CEMENTO
CESPED
SINTETICO
«enumerati... diaSemana
LUNES
MARTES
MIERCOLES
JUEVES
VIERNES
SABADO
DOMINGO
Complejo
- altura: int
- calle: char
- celular: int
- correo: char
- nombre: char
Localidad
- codigoPostal: int
- nombre: char
creat(fechaHoraInicio: datetime, fechaHoraFin: datetime, jugador: Jugador, cancha: Cancha, partido: Partido): Turno {
BEGIN
t += new Turno()
t.setFechaHoraInicio(fechaHoraInicio)
t.setFechaHoraFin(fechaHoraFin)
t.setJugador(jugador)
t.setCancha(cancha)
t.setPartido(partido)
return t
END
}
+excepciones
Excepcion
- fecha: date
- horaFin: time
- horaInicio: time
- motivo: char
estaCancelado(): boolean {
BEGIN
return self.cancelacion != null
END
}
Turno
- fechaHoraFin: datetime
- fechaHoraInicio: datetime
+ crear(datetime, datetime, Jugador, Cancha, Partido): Turno
+ estaCancelado(): boolean
+ puedeCancelarse(): boolean
+turno
+cancelacion 0..1
«enumerati... estadoCertificado
PENDIENTE
APROBADO
RECHAZADO
BusquedaCompañero
- estado: estadoBusqueda
- fechaHoraDesde: datetime
- fechaHoraHasta: datetime
Cancelacion
- fechaHora: datetime
+ crear(datetime, Turno, Jugador): Cancelacion
crear(fechaHora: datetime, turno: Turno, jugador: Jugador): Cancelacion {
BEGIN
c = new Cancelacion()
c.setFechaHora(fechaHora)
c.setTurno(turno)
c.setJugador(jugador)
turno.setCancelacion(c)
return c
END
}
«enumeration» estadoPartido
PENDIENTE
CANCELADO
CONFIRMADO
FINALIZADO
«enumeration» estadoBusqueda
ACTIVA
COMPLETADA
CANCELADA
Categoria
- nivel: int
Jugador
- ausencias: int
- contraseña: char
- correo: char
- foto: char
- nombre: char
- partidosGanados: int
- partidosPerdidos: int
- reputacion: int
+jugador
publica
realiza
posee
Recibe
Pone
Calificacion
- calificacion: int
Set
- gamesLocal: int
- gamesVisitante: int
- numero: int
+partido
Partido
- estado: estadoPartido
Certificado
- estado: estadoCertificado
- fechaEnvio: datetime
- motivoRechazo: char
- url: char
evalua
Administrador
- contraseña: char
- correo: char
- nombre: char
Pareja
- sumaCategorias: int
+parejaLocal
+parejaVisitante
publica
BusquedaMatch
- estado: estadoBusqueda
- fechaHoraDesde: datetime
- fechaHoraHasta: datetime
- sumaDeCategorias: int
hayDisponibilidad(cancha: Cancha, fechaHoraInicio: datetime, fechaHoraFin: datetime): boolean {
BEGIN
franja = cancha.buscarFranjaHabilitada(fechaHoraInicio, fechaHoraFin)
if franja == null
return false
if franja.tieneExcepcionEn(fechaHoraInicio, fechaHoraFin)
return false
if self.turnoBD.haySolapamiento(cancha, fechaHoraInicio, fechaHoraFin)
return false
return true
END
}
verificadorDisponibilidad
+ hayDisponibilidad(Cancha, datetime, datetime): boolean
+verificador
puedeCancelarse(): boolean {
BEGIN
return self.fechaHoraInicio > = (ahora() + 24 horas)
END
}
controllerTurno
+ altaTurno(datetime, datetime, Jugador, Cancha, Partido): char
+ cancelarTurno(Turno, Jugador): char
+turnoBD
turnoBD
+ haySolapamiento(Cancha, datetime, datetime): boolean
+ persistirTurno()
altaTurno(fechaHoraInicio: datetime, fechaHoraFin: datetime, jugador: Jugador, cancha: Cancha, partido: Partido): char {
Pre: partido.getEstado() == CONFIRMADO
BEGIN
if self.verificador.hayDisponibilidad(cancha, fechaHoraInicio, fechaHoraFin)
t = Turno.crear(fechaHoraInicio, fechaHoraFin, jugador, cancha, partido)
self.turnoBD.persistir(t)
return "turno creado con exito"
else
return "No hay disponibilidad en esa fecha y horario especifica"
END
}
cancelarTurno(turno: Turno, jugador: Jugador): char {
BEGIN
if turno == null
return "El turno no existe"
if turno.getJugador() != jugador
return "El jugador no puede cancelar este turno"
if turno.estaCancelado()
return "El turno ya fue cancelado"
if not turno.puedeCancelarse()
return "No se puede cancelar, faltan menos de 24hs para el turno"
c = Cancelacion.crear(ahora(), turno, jugador)
self.turnoBD.persistir(turno)
return "turno cancelado con exito"
END
}